

After changing the strings in `args` to lowercase, we pass the items in `args` and key-value pairs in `kwargs` as separate arguments to `print()` using the `*` and `**` syntax ❸. The return value of `print()` also gets returned as the return value of `printLower()`. These steps effectively wrap the `print()` function.

Functional Programming

Functional programming is a programming paradigm that emphasizes writing functions that perform calculations without modifying global variables or any external state (such as files on the hard drive, internet connections, or databases). Some programming languages, such as Erlang, Lisp, and Haskell, are heavily designed around functional programming concepts. Although not shackled to the paradigm, Python has some functional programming features. The main ones that Python programs can use are side-effect-free functions, higher-order functions, and lambda functions.

Side Effects

Side effects are any changes a function makes to the parts of the program that exist outside of its own code and local variables. To illustrate this, let's create a `subtract()` function that implements Python's subtraction operator `(-)`:

```
>>> def subtract(number1, number2):
...     return number1 - number2
...
>>> subtract(123, 987)
-864
```

This `subtract()` function has no side effects. That is, it doesn't affect anything in the program that isn't a part of its code. There's no way to tell from the program's or the computer's state whether the `subtract()` function has been called once, twice, or a million times before. A function might modify local variables inside the function, but these changes remain isolated from the rest of the program.